# An STM32F051 Serial Chess Server with Scratchpad Move Validation and a Qt 6 C++ GUI Client

Paul Kabulu (KBLPAU002) Mlungisi Sibeko (SBKMLU001)

EEE3095S CS Project 2026, Department of Electrical Engineering
University of Cape Town

***Abstract*—Relying on a graphical client to enforce game rules in a client-server setup makes it easy for bugs, dropped packets, or modified clients to desynchronise the game state. In this report, we present an embedded Chess server running on an STM32F051R8T6 microcontroller that communicates over a 115200-baud UART link with a lightweight Qt 6 C++ graphical user interface (GUI). The microcontroller maintains the single source of truth using a 64-byte board array and validates all piece geometry, path blocking, turn order, and king-safety rules through a six-stage check before updating the board's state. The server also supports move-highlight queries, four onboard pushbuttons for local mode selection and resignation, LED turn indicators, and a 1-ply greedy heuristic AI opponent. Serial messages use newline-delimited ASCII frames capped at 75 bytes, with a 500 ms inter-byte timeout to clear incomplete transmissions without blocking the main loop. We verified the system on physical STM32F051 hardware using an automated Python `pytest` and `pySerial` test suite, passing all 29 test cases across FSM transitions, move legality, endgames, AI behaviour, and serial fault recovery.**

***Index Terms*—Embedded Systems, Chess Engine, STM32F051, UART Protocol, Qt 6, Client-Server Architecture.**

## I. Introduction

In a client-server game architecture, letting the PC client validate moves or track the board state means that corrupted serial frames or client-side bugs can easily break the game [1]. To prevent this, the embedded server needs to act as the sole referee, treating every incoming serial message as an untrusted request. Our design makes sure that any malformed frame, out-of-turn command, or illegal Chess move is rejected with a specific negative acknowledgement (`NACK`) before the microcontroller modifies its internal board array.

While $3 \times 3$ Tic-Tsc-Toe is the standard baseline for the EEE3095S serial game server project [1], we chose to implement Chess ($8 \times 8$ board, six piece types, sliding-piece path checks, check/checkmate detection, and an on-board AI opponent [6], [9]) to properly test server-side validation on a resource-constrained 32-bit ARM Cortex-M0 microcontroller (STM32F051R8T6, 8 MHz clock, 8 KB SRAM, 64 KB Flash) [2]. On the PC side, we built a C++ GUI using Qt 6 (`QWidget`, `QPainter`, and event-driven `QSerialPort`) [3], alongside a Python 3 `pySerial` [4] and `pytest` [5] test suite for automated hardware-in-the-loop testing.

Our implementation is built around three main design choices:

1) **Bare-Metal Non-Blocking Firmware:** The firmware accesses the STM32F051 `USART1`, `GPIOA`/`GPIOB`, and Cortex-M0 `SysTick` registers directly [2]. Avoiding HAL overhead keeps the code compact while polling UART bytes, debouncing four pushbuttons, driving status LEDs, and tracking a 500 ms frame timeout inside a single main loop.
2) **Scratchpad King-Safety Check:** Because a move is illegal if it leaves the player's own King in check, candidate moves are first tested on a temporary 65-byte copy of the board (`test_board`) inside `try_move()`. The actual game board is only updated if all checks pass [7], [9].
3) **Thin Qt 6 Client with Handshake Verification:** The Qt 6 GUI contains no Chess rule logic. It verifies that the STM32 is powered and responsive using a 1500 ms `CONNECT` handshake timer, asks the MCU which squares a selected piece can move to (`MOVES:<sq>`), sends algebraic move requests (`MOVE:e2e4`), and animates piece movement by comparing consecutive 64-character board strings sent back by the server.

**Project Plan and Task Allocation.** The project schedule and task breakdown between team members are shown in Appendix A, Table I.

**Report Structure.** Section II defines the UART framing rules, 64-byte board encoding, command set, error priorities, and protocol extensions. Section III presents the modular architecture, server FSM, Qt 6 GUI and handshake flowcharts, and hardware circuit schematic. Section IV explains key firmware and C++ client code excerpts. Section V covers the six-stage move validation pipeline, state-invariance proof, and 1-ply AI opponent. Section VI examines protocol recovery from corrupted or incomplete frames. Section VII presents the automated `pytest` hardware results, Section VIII summarises the project's usefulness, limitations, and future work, and Sections IX–X provide links to the demonstration video and source repository.

## II. Requirements and Message Protocol

### A. Newline-Delimited ASCII Framing

Serial communication runs over the STM32F051 `USART1` peripheral on pins `PA9` (`TX`) and `PA10` (`RX`). On the UCT development board, these pins route directly to the onboard ST-Link Virtual COM Port (VCP), allowing a single USB cable to handle power, SWD flashing, and serial data at 115200 baud, 8N1, with no flow control [2].

All messages are plain ASCII strings ending with a line feed (`\n`, `0x0A`). To work cleanly across both Windows (`\r\n`) and Linux/macOS (`\n`) serial tools, the firmware simply ignores carriage returns (`\r`, `0x0D`). Frame length limits, timeouts, and UART error recovery rules are summarised in Appendix B, Table III.

### B. 64-Byte Board State Encoding

Instead of parsing variable-length Forsyth-Edwards Notation (FEN) strings on the microcontroller, the server represents the $8 \times 8$ board as a fixed 64-character ASCII string indexed from 0 to 63 in row-major order, starting at the top-left square (`a8` = index 0, `h1` = index 63). Uppercase letters represent White pieces (`P, N, B, R, Q, K`), lowercase letters represent Black pieces (`p, n, b, r, q, k`), and a period (`.`) represents an empty square.

For a square at row $r \in \{0..7\}$ (where $r = 0$ is rank 8 and $r = 7$ is rank 1) and column $c \in \{0..7\}$ (where $c = 0$ is file `a`), the array index $idx$ is calculated in `square_text_to_index()` as:

$$idx = 8r + c = 8(8 - \text{rank}) + (\text{file} - \text{`a'}) \tag{1}$$

For example, when White plays `MOVE:e2e4` (moving the pawn from index 52 to index 36), index 52 becomes `.`, index 36 becomes `P`, and the turn indicator updates to `TURN:B`.

### C. Command Set and Hardware Inputs/Outputs

The full serial command set is listed in Appendix B, Table IV. Alongside UART commands from the PC, the firmware polls four active-low onboard pushbuttons (`SW0`–`SW3` on `PA0`–`PA3`, configured with internal pull-up resistors in `GPIOA_PUPDR`) using a 50 ms debounce timer:

- **`SW0` (`PA0`):** Soft reset in the current game mode (Human vs. Human or Human vs. AI); sends `OK:BOARD:<64>:TURN:W\n`.
- **`SW1` (`PA1`):** Starts a new Human vs. Human game; sends `OK:BOARD:<64>:TURN:W\n` followed by `EVENT:NEW\n`.
- **`SW2` (`PA2`):** Starts a new Human vs. AI game; sends `OK:BOARD:<64>:TURN:W\n` followed by `EVENT:NEW_AI\n`.
- **`SW3` (`PA3`):** Hardware resign button during an active game; enters `STATE_GAME_OVER` and sends `OK:GAMEOVER:WIN:<Opponent>\n`.

Two user LEDs on `PB0` (`D0`) and `PB1` (`D1`) display the server state directly on the board: both LEDs are OFF in `STATE_IDLE`; `PB0` lights up during White's turn and `PB1` lights up during Black's turn in `STATE_IN_PROGRESS` (in Human vs. Human games); and both LEDs blink together at 2 Hz (every 250 ms) in `STATE_GAME_OVER`.

### D. Validation Priority and Protocol Extensions

Because a single bad command could fail multiple checks at once (for example, an out-of-bounds move sent when no game is running), the server checks incoming frames in a strict eight-step priority order. The first check that fails stops evaluation and returns the matching `NACK`, as shown in Appendix B, Table V. The connection handshake and message sequence between the PC client and STM32 server are illustrated in Appendix C, Figure 3.

To adapt the 9-cell Tic-Tac-Toe reference protocol [1] to Chess, we made four practical extensions: (1) Using 4-character algebraic source-destination payloads (`MOVE:e2e4`) instead of single cell numbers; (2) Splitting move errors into `NACK:EMPTY`, `NACK:NOTURN`, and `NACK:OCCUPIED` since Chess pieces belong to a specific player and move between squares; (3) Adding `NACK:ILLEGAL` for geometry/check violations alongside a `MOVES:<sq>` query so the GUI can highlight legal moves without knowing Chess rules; and (4) Appending `:TURN:<W|B>` and `EVENT:` messages so the GUI stays in sync with hardware button presses.

## III. Specification and Design

### A. System Architecture and Module Breakdown

We split the system cleanly between the STM32F051 firmware and the PC client, as summarised in Appendix A, Table II. All game state, turn tracking, legal move generation, AI evaluation, and rule checks happen on the microcontroller. The PC client only handles serial communication, handshake timeouts, and drawing the board.

### B. Server Finite-State Machine (FSM)

The server FSM is shown in Appendix C, Figure 1. It has three main states: `STATE_IDLE`, `STATE_IN_PROGRESS`, and `STATE_GAME_OVER`. When a `MOVE:<s1><s2>` command arrives while in `STATE_IN_PROGRESS`, the server runs it through the six-stage validation pipeline before updating the board or checking for game-over conditions.

### C. Event-Driven Qt 6 Client and Handshake Logic

On Linux and macOS, opening a serial port (`/dev/ttyACM0`) succeeds as soon as the USB-to-UART bridge chip is plugged in, even if the STM32 microcontroller itself has no power or is held in reset. To avoid showing a false "Connected" state, the GUI performs an active handshake when the user clicks **Connect** (Appendix C, Figure 3): it opens `QSerialPort`, places the UI in a pending state, starts a 1500 ms single-shot timer, and sends `CONNECT\n`. Only when the STM32 replies with `OK` does the GUI unlock the game controls; if the timer expires first, the port closes and reports a connection error. The full GUI application workflow is shown in Appendix C, Figure 2.

### D. Hardware Circuit Connections

The circuit diagram for our setup on the UCT STM32F051 development board is shown in Appendix C, Figure 4. A single USB cable supplies +5V power (regulated to +3.3V by the onboard MCP1702 LDO), provides the SWD programming interface, and carries the 115200-baud UART link between PA9/PA10 and the host PC via the onboard ST-Link V2.1 bridge. Pushbuttons SW0–SW3 on PA0–PA3 pull the inputs to ground when pressed, while status LEDs D0 and D1 on PB0 and PB1 are driven through $150\,\Omega$ current-limiting resistors.

## IV. Implementation

### A. Non-Blocking UART Polling and Timeout Recovery

Instead of using blocking HAL receive calls that could freeze button polling or LED blinking, our main loop checks the USART1_ISR_RX_NOT_EMPTY register flag directly alongside a 1 kHz SysTick millisecond counter (Appendix D, Listing 1). If hardware overrun (ORE), framing (FE), or noise (NE) flags are triggered during rapid serial bursts, the loop clears them immediately via USART1_ICR so the UART receiver never locks up.

### B. Scratchpad Move Validation

In Chess, a move that looks geometrically valid is still illegal if it leaves the player's King in check (for example, moving a pinned piece) [7], [9]. As shown in Appendix D, Listing 2, try_move() copies the current board into a local 65-byte array (test_board), makes the move there, and calls is_king_in_check(). Passing commit = false lets us reuse the exact same function to check for legal moves during checkmate/stalemate scans, MOVES:<sq> queries, and AI move selection without touching the real board. When commit = true, it also updates the 50-move draw counter (halfmove_clock), resetting it on pawn moves or captures.

### C. Qt 6 Serial Line Parser and Handshake Timeout

The Qt 6 client reads incoming serial bytes asynchronously whenever the QSerialPort::readyRead signal fires, buffering partial chunks until a newline (\n) arrives, while a 1500 ms timer guards the initial CONNECT handshake (Appendix D, Listing 3). When a new board string arrives, the GUI compares the old and new 64-byte strings and smoothly animates the moving piece at 60 FPS using a cubic ease-out curve.

## V. Game Logic and Server Authority

### A. Six-Stage Move Validation Pipeline

Every MOVE:<s1><s2> command is converted into array indices (0..63) using Equation (1) and checked in six steps inside try_move():

1) **Bounds Check:** Rejects squares outside a1–h8 with NACK:RANGE.
2) **Empty Source Check:** Rejects empty starting squares (.) with NACK:EMPTY.
3) **Turn Check:** Rejects attempts to move the opponent's pieces with NACK:NOTURN.
4) **Friendly Fire Check:** Rejects moves that land on the player's own pieces with NACK:OCCUPIED.
5) **Piece Geometry and Path Blocking:** Checks $\Delta r$ and $\Delta c$ for each piece type in can_piece_move(). For sliding pieces (Rooks, Bishops, Queens), is_path_clear() steps along the ray to make sure no pieces are in the way. Pawns check for 1- or 2-square forward moves onto empty squares or 1-square diagonal captures, automatically promoting to a Queen on the back rank (row 0 or 7).
6) **King Safety:** Simulates the move on test_board[65] and scans all enemy pieces in is_king_in_check() to make sure the active player's King cannot be captured.

### B. Why Rejected Commands Never Corrupt the Board

Tracing the execution paths through the firmware shows why rejected requests never alter the active game state:

- **Framing Errors (NACK:TIMEOUT, NACK:BADCMD):** Caught in the main UART receive loop before the command parser is even called.
- **State Errors (NACK:NOGAME):** Checked at the top of each command handler before any board coordinates are read.
- **Move Errors:** Inside try_move() (Listing 2), the real game->board array is only read, never written, during stages 1–6. The memcpy() that overwrites game->board and flips game->active_turn only runs at the very end of the function if commit == true and all six checks have passed.

### C. Endgame Detection and 1-Ply Greedy AI

After every valid move, player_has_legal_moves() loops over all 64 squares and tests every possible destination ($64 \times 64 = 4{,}096$ pairs max) using try_move(g, s, d, false). If the next player has zero legal moves, the server checks is_king_in_check(): if the King is in check, it declares **Checkmate** (WIN:<W|B>); otherwise, it declares **Stalemate** (DRAW). The server also declares a draw automatically if only the two Kings remain on the board (has_only_kings_left()) or if 50 full moves pass without a pawn move or capture (halfmove_clock >= 100).

For the AI opponent (NEW:AI), running a multi-ply recursive Minimax search with Alpha-Beta pruning [6], [8] is impractical on an 8 MHz Cortex-M0 with only 8 KB of SRAM, as Chess's branching factor ($b \approx 35$) quickly risks stack overflow and noticeable serial lag. Instead, we implemented a fast **1-ply greedy heuristic evaluator** (make_computer_move()). It tests all legal Black moves via try_move(g, from, to, false) and scores captures using standard piece values (P $= 1$, N/B $= 3$, R $= 5$, Q $=$ 9, multiplied by 10) [7], plus a $+1$ positional bonus for moves landing on the central $4 \times 4$ squares ($r, c \in \{2..5\}$). Because every AI candidate is filtered through try_move(), the AI can never play an illegal move.

## VI. Protocol Robustness and Integrity

The server handles noisy serial links, oversized payloads, and dropped connections without hanging or corrupting memory. The specific recovery mechanisms and matching `pytest` tests are listed in Appendix E, Table VI.

Furthermore, the firmware cannot get stuck in an infinite loop: every loop in the engine has a fixed upper bound (at most $4,096$ move pairs for checkmate scans and at most 7 steps for sliding raycasts), and sending an 81-byte board reply at 115200 baud takes just 7.0 ms. Because UART reception is polled non-blockingly and incomplete lines time out after 500 ms via `SysTick`, the main loop always stays responsive.

## VII. Validation

We verified the system on physical STM32F051 hardware using an automated Python test script (`test_chess.py`) built with `pytest` and `pySerial`. The script opens the ST-Link Virtual COM Port at 115200 8N1 and runs 29 test cases covering handshakes, legal/illegal moves, move highlighting, Fool's Mate, stalemate, insufficient material, serial framing faults, and the AI opponent. Appendix E, Table VII summarises the test suite, and Listing 4 shows the terminal output with all 29 tests passing in 3.06 seconds.

Running hardware-in-the-loop tests helped us catch and fix three practical serial issues:

1) **ST-Link VCP DTR/RTS Boot Glitches:** Opening the serial port from Python toggles the `DTR`/`RTS` lines, which can reset the STM32 or inject a stray byte into the UART RX buffer. We fixed this in `test_chess.py` by disabling `dtr`/`rts` on port open and sending warm-up `DISCONNECT\n` commands before starting assertions.
2) **Back-to-Back Command Collisions:** Early versions of our GUI sent `DISCONNECT\n` immediately followed by `NEW\n` with no delay, causing bytes to arrive while the MCU was still sending back `OK\n`. Removing the redundant `DISCONNECT` call made starting a new game instant and reliable.
3) **USB Bridge vs. MCU Power:** Because `QSerialPort::open()` succeeds even when the STM32 is unpowered (since the ST-Link USB chip is powered by the PC), adding the 1500 ms `CONNECT` $\rightarrow$ `OK` handshake timeout prevented the GUI from showing a false connected status.

## VIII. Conclusion

We designed, built, and tested an STM32F051 serial Chess server paired with a thin Qt 6 C++ GUI and a 29-test `pytest` hardware verification suite. Keeping all rule enforcement, check detection, move highlighting, and 1-ply AI evaluation on the microcontroller - and testing moves on a temporary scratchpad board before committing them - ensures that invalid client requests cannot corrupt the game state.

**Usefulness.** The core techniques developed here -validating moves on a temporary copy before updating state, using a serial handshake to confirm device presence, and handling UART timeouts without blocking - are directly useful in practical embedded systems where a microcontroller must verify PC commands before controlling hardware.

**Limitations and Possible Improvements.** To keep the firmware simple, fast, and well within the 8 KB SRAM limit, we omitted castling, *en passant*, and threefold repetition, implementing automatic Queen promotion, insufficient-material detection, and the 50-move draw rule instead. Future improvements could expand the 1-ply greedy AI to a 2-ply Minimax search with Alpha-Beta pruning [8], [10] and use an STM32 hardware timer to implement a per-player Chess clock.

## IX. Video

The demonstration video showing the hardware setup, serial handshake, move highlighting, `NACK` rejections, Fool's Mate, onboard `SW0`–`SW3` pushbuttons / LED indicators, and AI opponent is available at: https://youtu.be/YgMaVd8k1dc

## X. Code

The complete STM32F051 firmware, Qt 6 C++ GUI client, and 29-test `pytest` suite (along with setup, build, and execution instructions in `README.md`) are available at: https://github.com/paulkabzz/chess-server

## References


[1] Department of Electrical Engineering, "EEE3095S CS Project 2026: Serial Game Server," University of Cape Town, Course Brief, 2026.
[2] STMicroelectronics, *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 Advanced ARM-Based 32-Bit MCUs Reference Manual*, Rev. 10, 2021.
[3] The Qt Company, "Qt 6 Documentation: `QWidget`, `QPainter`, and `QSerialPort`," 2026.
[4] C. Liechti, "pySerial Documentation (v3.5)," 2020.
[5] H. Krekel *et al.*, "pytest: helps you write better programs," 2026.
[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
[7] Chess Programming Wiki, "Point Value, Move Generation, and 1-Ply Search," 2026.
[8] S. Lague, "Algorithms Explained — minimax and alpha-beta pruning," YouTube, 2018.
[9] S. Lague, "Coding Adventure: Chess," YouTube, 2021.
[10] S. Lague, "Coding Adventure: Making a Better Chess Bot," YouTube, 2023.

# APPENDIX A
# PROJECT PLAN AND ARCHITECTURE

TABLE I
PROJECT MILESTONES AND TASK ALLOCATION (DELIVERABLE A.1).

| Milestone | Planned Deliverables | Lead |
|---|---|---|
| M1: Spec | Define 64-byte ASCII board encoding, command/NACK grammar, and FSM transitions. | Both |
| M2: Core | Configure USART1 (115200 8N1), SysTick timer, 500 ms frame timeout, and GPIO LEDs/buttons. | Paul |
| M3: Engine | Implement 6-piece geometry, sliding raycasts, check/checkmate detection, and 1-ply greedy AI. | Paul |
| M4: GUI | Build Qt 6 `BoardWidget` with ease-out animations, `QSerialPort` handshake, and modals. | Mlungisi |
| M5: Harness | Develop 29-test `pytest` suite (`test_chess.py`) covering FSM, rules, endgames, and AI. | Mlungisi |
| M6: Report | Hardware-in-the-loop verification, demo video recording, and final report. | Both |

TABLE II
SYSTEM MODULE PARTITIONING ACROSS STM32 SERVER AND PC CLIENT (DELIVERABLE A.4).

| Subsystem | Target | Primary Responsibilities |
|---|---|---|
| UART/SysTick | STM32 | Direct-register `USART1` polling, error flag clearing, 75B buffer, and 500 ms timeout via 1 kHz `SysTick`. |
| Server FSM | STM32 | Enforces `IDLE`, `IN_PROGRESS`, and `GAME_OVER` states; drives `PB0`/`PB1` LEDs and `SW0`–`SW3` buttons. |
| Chess Engine | STM32 | 64-byte board array, 6-stage scratchpad validator (`try_move`), checkmate/draw scanner, and 1-ply AI. |
| Qt 6 GUI | PC | $8 \times 8$ canvas painter, 60 FPS move animator, `QSerialPort` 1.5 s handshake check, and game-over modal. |
| pytest Suite | PC | Automated `test_chess.py` harness running 29 hardware-in-the-loop test vectors over serial. |

# APPENDIX B
# PROTOCOL SPECIFICATIONS

TABLE III
FRAMING RULES AND DETERMINISTIC SERVER RESPONSES (DELIVERABLE A.2).

| Rule | Server Response & Handling |
|---|---|
| Terminator | Line feed (`\n`, `0x0A`) ends a frame; optional preceding `\r` (`0x0D`) is ignored. |
| Max Length | $>$ 75 payload bytes latches `buffer_overflowed`, drops bytes until `\n`, and replies `NACK:BADCMD`. |
| Timeout | $>$ 500 ms with buffered bytes and no `\n` clears the buffer and replies `NACK:TIMEOUT`. |
| Empty Line | Empty frame (`\n` with `buffer_index == 0`) replies `NACK:BADCMD`. |
| UART Flags | Overrun (`ORE`), framing (`FE`), and noise (`NE`) flags in `USART1_ISR` are cleared via `USART1_ICR`. |

TABLE IV
COMMAND SET AND ASYNCHRONOUS HARDWARE EVENTS (DELIVERABLE A.2).

| Command / Event | Valid In | Server Response |
|---|---|---|
| `CONNECT` | All | `OK\n` (resets session to `STATE_IDLE`, turns both LEDs OFF) |
| `NEW` | All | `OK:BOARD:<64>:TURN:W\n` (2-player mode, enters `IN_PROGRESS`, `PB0` ON) |
| `NEW:AI` | All | `OK:BOARD:<64>:TURN:W\n` (enables STM32 1-ply AI for Black) |
| `MOVE:<s1><s2>` | `IN_PROG` | `OK:BOARD:<64>:TURN:<W\|B>\n` or terminal `WIN:<W\|B>:BOARD...` / `DRAW:BOARD...` |
| `MOVES:<sq>` | `IN_PROG` | `VALID:<sq1>,<sq2>,...\n` (lists legal target squares for piece at `<sq>`) |
| `LOAD:<64>:<T>` | All | `OK:BOARD:<64>:TURN:<T>\n` (loads custom board state for testing) |
| `STATE` | Active/Over | `BOARD:<64>:TURN:<W\|B>\n` |
| `RESIGN` | `IN_PROG` | `OK:GAMEOVER:WIN:<B\|W>\n` (enters `STATE_GAME_OVER`, blinks `PB0`/`PB1`) |
| `DISCONNECT` | All | `OK\n` (returns server to `STATE_IDLE`) |
| `SW0`–`SW2` Press | All | `OK:BOARD:<64>:TURN:W\n` (plus `EVENT:NEW` for `SW1` or `EVENT:NEW_AI` for `SW2`) |
| `SW3` Press | `IN_PROG` | `OK:GAMEOVER:WIN:<B\|W>\n` (hardware resign button) |

# APPENDIX C
# SYSTEM DESIGN AND CIRCUIT DIAGRAMS

# APPENDIX D
# CODE EXCERPTS

TABLE V
ERROR RESPONSES IN STRICT EVALUATION PRIORITY ORDER (DELIVERABLE A.2).

| Priority & Response | Triggering Condition |
|---|---|
| 1. `NACK:TIMEOUT` | Partial frame sits in RX buffer for $> 500$ ms without a terminating `\n`. |
| 2. `NACK:BADCMD` | Frame exceeds 75 bytes, is empty, or has an unrecognised command verb. |
| 3. `NACK:NOGAME` | `MOVE`, `MOVES`, `STATE`, or `RESIGN` sent in `IDLE`, or move/resign sent in `GAME_OVER`. |
| 4. `NACK:RANGE` | Coordinates fall outside `a1`–`h8` or payload length $\neq 4$ ($\neq 2$ for `MOVES:`). |
| 5. `NACK:EMPTY` | Source square contains no piece (`.`). |
| 6. `NACK:NOTURN` | Source square holds a piece belonging to the inactive player. |
| 7. `NACK:OCCUPIED` | Target square is occupied by one of the active player's own pieces. |
| 8. `NACK:ILLEGAL` | Move violates piece geometry, path is blocked, or leaves the active player's King in check. |

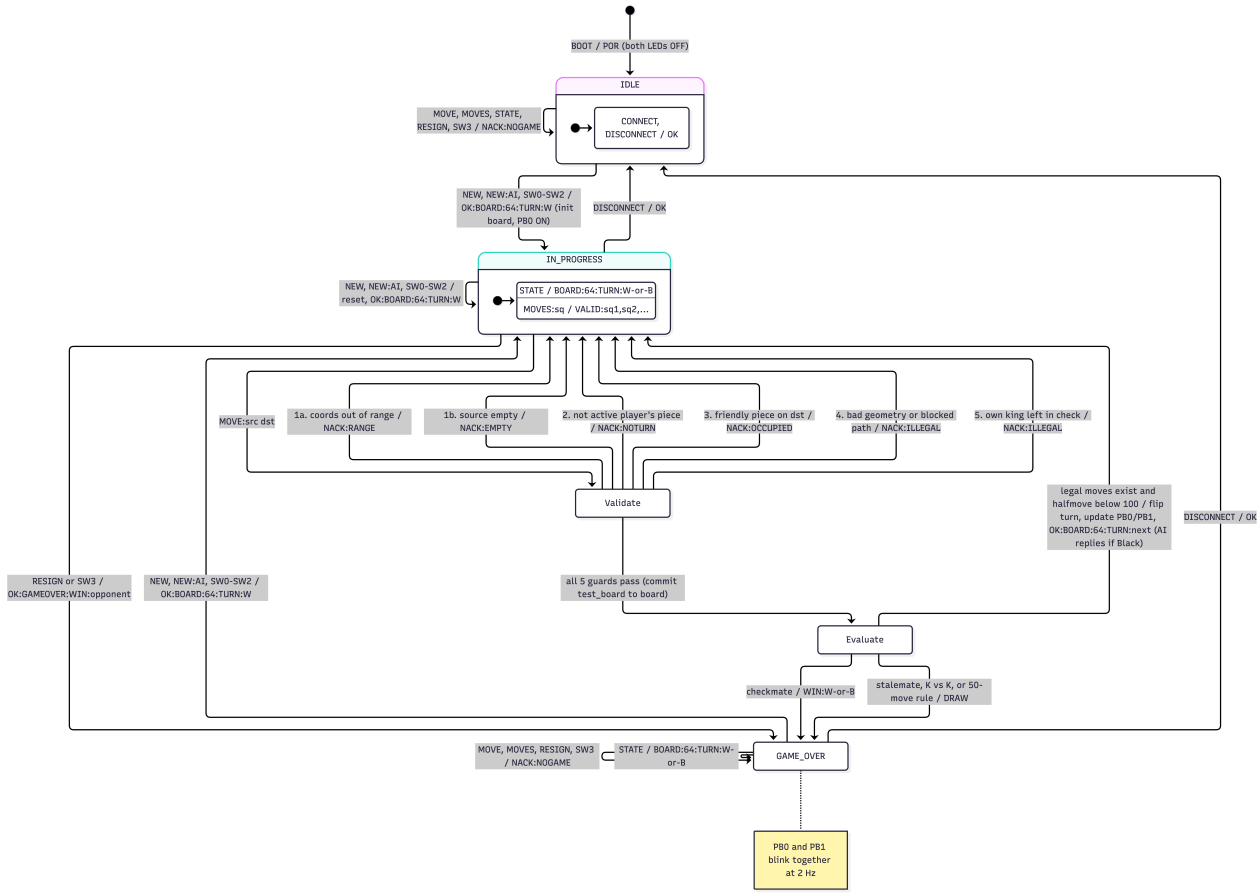



Fig. 1. Embedded Server FSM and 6-stage move-validation branch (Deliverable A.5).

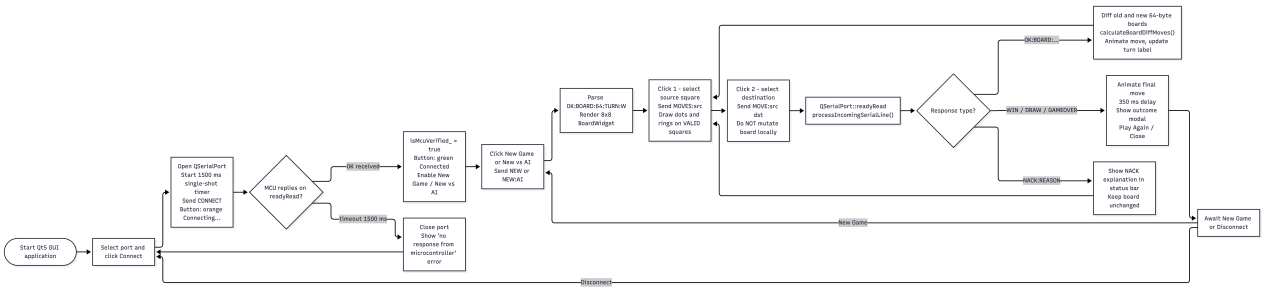



Fig. 2. Qt 6 C++ GUI application flowchart (Deliverable A.5).

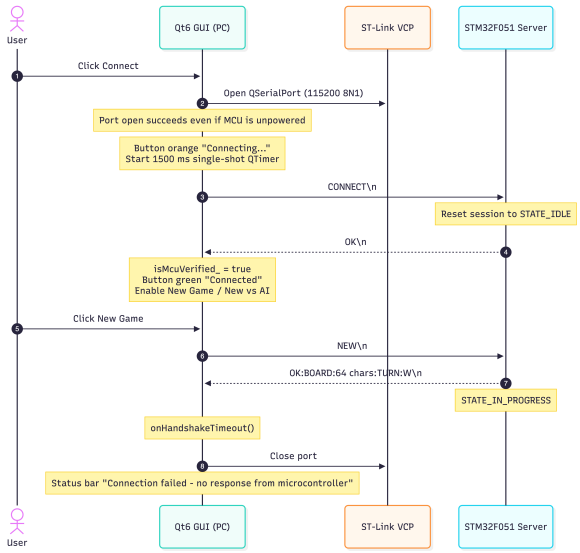



Fig. 3. Application-layer serial handshake and protocol sequence diagram (Deliverable A.2).

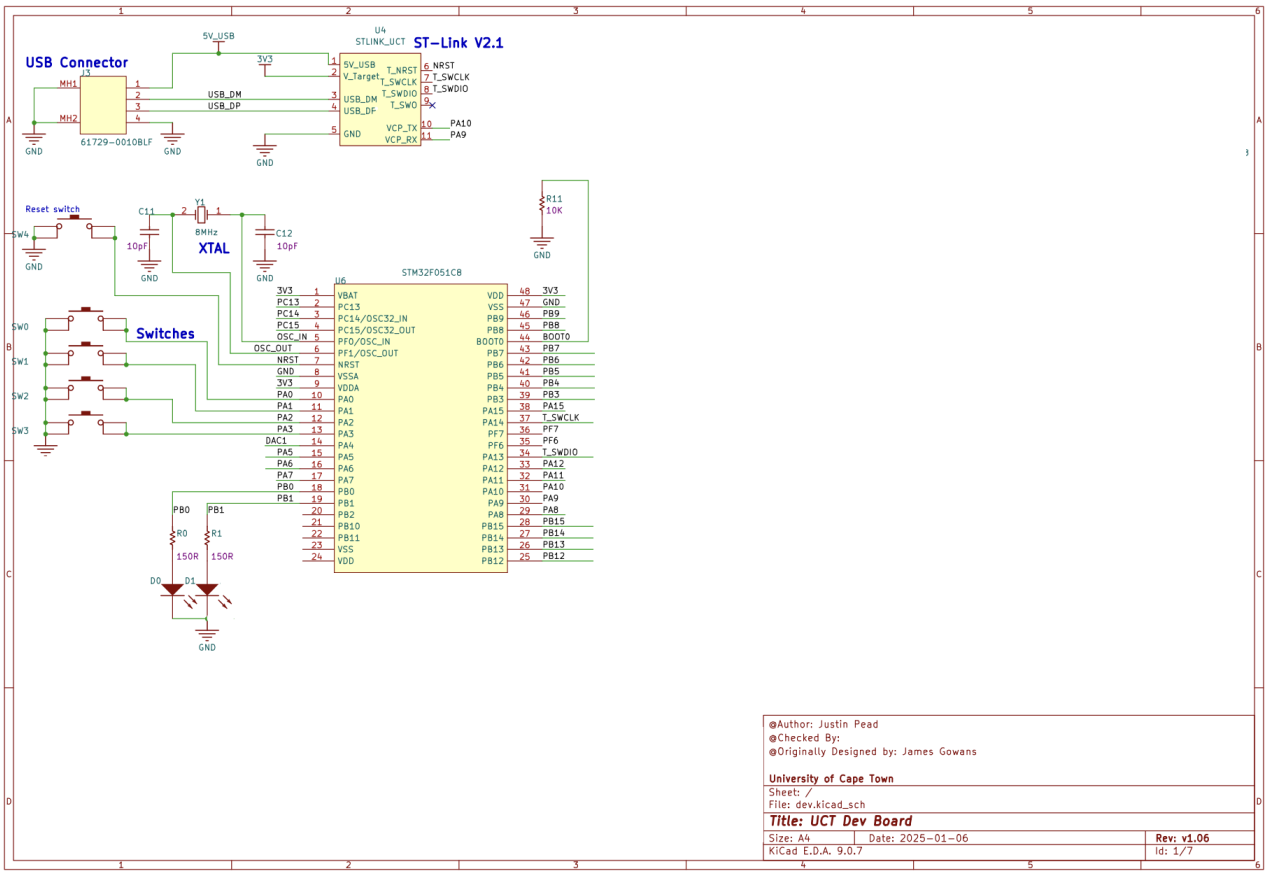



Fig. 4. Hardware circuit schematic showing the STM32F051, onboard ST-Link V2.1 Virtual COM Port (PA9/PA10), 3.3V regulator, pushbuttons SW0–SW3 (PA0–PA3), and status LEDs D0–D1 (PB0–PB1) (Deliverable A.3).

```
if (USART1_ISR & USART1_ERROR_FLAGS_MASK) {
    USART1_ICR = USART1_ERROR_FLAGS_MASK;
}
if (USART1_ISR & USART1_ISR_RX_NOT_EMPTY) {
    char incoming_char = (char)(USART1_RDR & USART1_RDR_BYTE_MASK);
    last_byte_received_time = millisecond_ticks;
    if (incoming_char == '\r') {
        // Ignore carriage return
    } else if (incoming_char == '\n') {
        if (buffer_overflowed || buffer_index == 0) {
            uart_send_string("NACK:BADCMD\n");
        } else {
            receive_buffer[buffer_index] = '\0';
            handle_uart_command(receive_buffer);
        }
        buffer_index = 0;
        buffer_overflowed = false;
    } else {
        if (buffer_index < UART_MAX_COMMAND_LENGTH) {
            receive_buffer[buffer_index++] = incoming_char;
        } else {
            buffer_overflowed = true;
        }
    }
}
if ((buffer_index > 0 || buffer_overflowed) &&
    (millisecond_ticks - last_byte_received_time > UART_RX_TIMEOUT_MS)) {
    buffer_index = 0;
    buffer_overflowed = false;
    uart_send_string("NACK:TIMEOUT\n");
}
```

Listing 1. Non-blocking USART1 byte accumulator and 500 ms timeout (`main.c`).

```
static MoveStatus_t try_move(ChessGame_t *game, int from_sq, int to_sq, bool commit) {
    if (from_sq < 0 || from_sq >= 64 || to_sq < 0 || to_sq >= 64) return ERR_RANGE;
    char piece = game->board[from_sq];
    if (piece == '.') return ERR_EMPTY;
    if (!belongs_to_player(piece, game->active_turn)) return ERR_NOTURN;

    char target = game->board[to_sq];
    if (target != '.' && belongs_to_player(target, game->active_turn)) return ERR_OCCUPIED;
    if (!can_piece_move(game->board, from_sq, to_sq)) return ERR_ILLEGAL;

    char test_board[65];
    memcpy(test_board, game->board, 65);
    move_piece_on_board(test_board, from_sq, to_sq);
    if (is_king_in_check(test_board, game->active_turn)) return ERR_ILLEGAL;

    if (commit) {
        if (to_uppercase_piece(piece) == 'P' || target != '.') {
            game->halfmove_clock = 0;
        } else {
            game->halfmove_clock++;
        }
        memcpy(game->board, test_board, 65);
        game->active_turn = (game->active_turn == 'W') ? 'B' : 'W';
    }
    return MOVE_OK;
}
```

Listing 2. Scratchpad move validation and king-safety check (`main.c`).

```
void MainWindow::readSerialData() {
    rxBuffer.append(serialPort->readAll());
    while (rxBuffer.contains('\n')) {
        int lineEnd = rxBuffer.indexOf('\n');
        QString line = QString::fromUtf8(rxBuffer.left(lineEnd)).trimmed();
        rxBuffer.remove(0, lineEnd + 1);
        if (!line.isEmpty()) processCommand(line);
    }
}
```

```cpp
void MainWindow::onHandshakeTimeout() {
    if (!isAwaitingHandshake || isMcuVerified) return;
    isAwaitingHandshake = false;
    serialPort->close();
    updateConnectionVisuals();
    statusBarLabel->setText("Connection failed -- no response from microcontroller.");
}
```

Listing 3. Qt 6 C++ serial frame parser and 1500 ms MCU handshake timeout (`mainwindow.cpp`).

## APPENDIX E
## VALIDATION RESULTS

TABLE VI
FAULT-HANDLING MECHANISMS AND VERIFICATION TESTS.

| Fault Condition | Recovery Mechanism | Test Function |
|---|---|---|
| Partial frame (no `\n`) | 500 ms `SysTick` timeout resets RX buffer and sends `NACK:TIMEOUT`. | `test_incomplete_frame_timeout` |
| Oversized frame ($>$ 75 B) | `buffer_overflowed` latches, drops bytes until `\n`, sends `NACK:BADCMD`. | `test_oversized_frame_rejected_and_recovers` |
| Empty line (`\n` only) | Detects `buffer_index == 0` on `\n` and replies `NACK:BADCMD`. | `test_empty_line_rejected` |
| Bad Command | Unrecognised verb rejected with `NACK:BADCMD`; FSM state unchanged. | `test_bad_command` |
| UART Hardware Error | Clears `ORE`/`FE`/`NE` flags in `USART1_ISR` via `USART1_ICR`. | All 29 burst serial tests |

TABLE VII
AUTOMATED VALIDATION TESTS EXECUTED ON PHYSICAL STM32F051.

| Test Category | pytest Functions (`test_chess.py`) | Result |
|---|---|---|
| Handshake & IDLE Guard | `test_connect_returns_ok`, `test_state_in_idle` | PASS |
| FSM Lifecycle | `test_new_returns_initial_board`, `test_disconnect_returns_to_idle` | PASS |
| Legal Move Execution | `test_legal_pawn_move` | PASS |
| Empty & Bounds Guards | `test_empty_source`, `test_out_of_range` | PASS |
| Turn & Occupancy Guards | `test_wrong_side_piece`, `test_own_piece_target` | PASS |
| Geometry & Raycast Guards | `test_illegal_geometry`, `test_blocked_slider` | PASS |
| State Invariance | `test_rejected_move_leaves_state_unchanged` | PASS |
| Move Highlighting (`MOVES:`) | `test_moves_query_initial_pawn`, `test_moves_query_blocked_rook`, `test_moves_query_bad_square` | PASS |
| Checkmate & Post-Mate Lock | `test_fools_mate`, `test_move_after_mate_rejected` | PASS |
| Resignation & Post-Game State | `test_resign`, `test_resign_after_white_move`, `test_state_after_game_over_report_board` | PASS |
| Draw Evaluation (`DRAW:`) | `test_stalemate_via_load`, `test_insufficient_material_via_load` | PASS |
| Framing & Error Recovery | `test_oversized_frame_rejected_and_recovers`, `test_incomplete_frame_timeout`, `test_empty_line_rejected`, `test_bad_command` | PASS |
| 1-Ply AI Engine | `test_ai_makes_move`, `test_ai_never_returns_illegal`, `test_ai_punishes_passive_move` | PASS |

```
$ make test
Running pytest suite on port /dev/ttyACM0...
============================= test session starts ==============================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.7.0
collected 29 items
test_chess.py::test_connect_returns_ok PASSED                    [  3%]
test_chess.py::test_state_in_idle PASSED                         [  6%]
test_chess.py::test_new_returns_initial_board PASSED             [ 10%]
...
test_chess.py::test_ai_makes_move PASSED                         [ 93%]
test_chess.py::test_ai_never_returns_illegal PASSED              [ 96%]
test_chess.py::test_ai_punishes_passive_move PASSED              [100%]
============================== 29 passed in 3.06s ==============================
```

Listing 4. Captured pytest execution log on physical STM32F051.